Sorting Algorithms: Algorithms, Advantages, Disadvantages and Real-World Uses

Based on the Java programs in package Sorting_Tasks: Bubble, Selection, Insertion and Merge Sort, each sorting a student-marks array in ascending order

1. Bubble Sort

Bubble_Sort.java | bubble_sort(int[] marks)

Algorithm (according to your program)

1. Start with the array **marks** of size n.
2. Repeat the outer loop for **i = 0** to **n-2** (one pass each).
3. In each pass, run the inner loop for **j = 0** to **n-i-2**.
4. Compare **marks[j]** with **marks[j+1]**.
5. If marks[j] > marks[j+1], swap them using a temporary variable (dummy).
6. After each pass the largest remaining value moves to the end of the unsorted part.
7. When all passes finish, the array is in ascending order; call **printing()** to display it.

Time: O(n^2) best/avg/worst (no early-exit in this code) | Space: O(1) | Stable: Yes

Advantages

+ Simplest sorting logic - easy to write, read and debug.
+ In-place: needs only O(1) extra memory (one temp variable).
+ Stable: equal marks keep their original order.
+ Easy to visualise because only neighbouring items are compared.

Disadvantages

- Very slow on large data: about $n^2/2$ comparisons even if the array is already sorted.
- Performs many swaps (writes) compared with selection sort.
- Your version has no "stop if no swap" check, so it never finishes early.

Where it is used (real world)

- Teaching sorting basics and algorithm analysis in classrooms.
- Very small lists (e.g. sorting 10-20 marks) where speed does not matter.
- Simple embedded/low-memory code and graphics demos where simplicity wins.

2. Selection Sort

Selection_Sort.java | selection_sort(int[] marks)

Algorithm (according to your program)

1. Start with the array **marks** of size n.
2. Repeat for **i = 0** to **n-2**; set **minIndex = i**.
3. Scan **j = i+1** to **n-1**; if marks[j] < marks[minIndex], set minIndex = j.
4. After the scan, swap marks[i] with marks[minIndex] using a temp variable.
5. The sorted part on the left grows by one element each pass.
6. When i reaches n-2, the whole array is ascending; call **printing()**.

Time: O(n^2) best/avg/worst | Space: O(1) | Stable: No | Swaps: at most n-1

Advantages

+ Makes the fewest swaps - at most n-1 writes in total.
+ In-place with O(1) extra memory.
+ Simple logic and very predictable running time.
+ Good when writing to memory is more costly than reading.

Disadvantages

- Always O(n^2) - no speed-up even when the data is already sorted.
- Not stable: the swap can change the order of equal marks.
- Too slow for large datasets such as millions of records.

Where it is used (real world)

- Flash / EEPROM and other devices where each write wears the memory.
- Small datasets and embedded systems with tight memory.
- Finding the k smallest items (e.g. the 3 lowest marks) without sorting everything.
- Teaching the idea of "select the minimum, place it first".

3. Insertion Sort

Insertion_Sort.java | insetion_sort(int marks[])

Algorithm (according to your program)

1. Start with the array **marks** of size n; treat marks[0] as already sorted.
2. Repeat for **i = 1** to **n-1**.
3. Store the current value: **key = marks[i]**, and set **j = i-1**.
4. While j >= 0 and key < marks[j], shift marks[j] one place right (marks[j+1] = marks[j]) and decrease j.
5. Insert the key into the gap: **marks[j+1] = key**.
6. Repeat until i reaches n-1; the array is now ascending; call **printing()**.

Time: best O(n), avg/worst O(n^2) | Space: O(1) | Stable: Yes

Advantages

+ Very fast on small or nearly-sorted data (best case O(n)).
+ Stable and in-place with O(1) extra memory.
+ Online: can sort data as it arrives, one item at a time.
+ Low overhead, so it beats complex sorts on tiny arrays.

Disadvantages

- O(n^2) on average and worst case (e.g. reverse-ordered data).
- Many element shifts when the data is far from sorted.
- Not suitable for large, random datasets.

Where it is used (real world)

- Adding new records into an already sorted list (e.g. new student marks).
- Sorting small arrays, like a hand of playing cards.
- Final stage inside hybrid sorts such as Timsort and introsort for small chunks.
- Real-time systems receiving a stream of values.

4. Merge Sort

Merge_Sort.java | mergeSort() and merge()

Algorithm (according to your program)

1. Call **mergeSort(arr, 0, n-1)**.
2. Base case: if **left >= right** (0 or 1 element), return.
3. Find the middle: **mid = left + (right-left)/2**.
4. Recursively sort the left half (left..mid) and the right half (mid+1..right).
5. **merge()**: copy the two halves into temporary arrays L and R.
6. Compare L[i] and R[j]; place the smaller into arr[k] (L is taken when L[i] <= R[j]); advance the pointers.
7. Copy any leftover elements of L or R into arr.
8. Print the original and sorted arrays with **printArray()**.

Time: O(n log n) best/avg/worst | Space: O(n) | Stable: Yes

Advantages

+ Guaranteed O(n log n) in every case - predictable and fast.
+ Stable (the <= comparison keeps equal items in order).
+ Handles very large datasets well; easy to run in parallel.
+ Works naturally with linked lists and external (disk) sorting.

Disadvantages

- Needs O(n) extra memory for the L and R temporary arrays.
- Recursion and copying add overhead, so it is slower than insertion sort on tiny arrays.
- More complex to write and understand than the simple sorts.

Where it is used (real world)

- Databases and big-data tools sorting files too large for RAM (external merge sort).
- Library sorts for objects: Java's Arrays.sort(Object[]) and Python's sorted() use merge-based Timsort.
- Sorting records by price/date/marks where equal items must keep their order.
- Merging sorted log files; counting inversions.

Algorithm	Best	Average	Worst	Extra space	Stable	In-place	Ops for n = 1000 (approx.)
Bubble Sort	O(n^2)*	O(n^2)	O(n^2)	O(1)	Yes	Yes	~500,000 comparisons
Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1)	No	Yes	~500,000 comparisons, <= 999 swaps
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes	Yes	~250,000 shifts (random data)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	No	~10,000 comparisons

** Bubble Sort here has no early-exit check, so even a sorted array costs O(n^2). With a "no swaps in a pass" flag its best case becomes O(n). n = array length (1,000 marks in the programs).*